

秒杀系统性能优化记录

优化原则：压测 → 定位瓶颈 → 根因分析 → 优化 → 重测验证。每次只改一个问题，可追溯。

优化 1：调高 /order 接口限流阈值

发现问题 (2026-06-21 Phase 1 初测)

并发	成功QPS	失败率	根因
10	99	60%	限流 + 重复购买
50	0	100%	@RateLimit(rate=100) 拦截全部请求
100	0	100%	同上
200	0	100%	同上

限流拦截日志：5,115 次。秒杀接口在 c≥50 时完全不可用。

根因分析

OrderController.java:28 — @RateLimit(rate=100, rateInterval=1000, message="秒杀人数过多")

100 QPS 限流阈值对秒杀业务严重偏低。真实秒杀场景中，活动开始瞬间的流量洪峰远超 100 QPS。限流器使用 Redis INCR + 固定 TTL 实现，非真正滑动窗口，存在窗口边界的误杀。

优化措施

```
// OrderController.java - 限流 100/s → 1000/s
- @RateLimit(rate = 100, rateInterval = 1000, ...)
+ @RateLimit(rate = 1000, rateInterval = 1000, ...)

// ProductController.java - 同步调整
// /product/detail: 500/s → 2000/s
// /product/active: 300/s → 1000/s
```

重测验证

并发	优化前成功QPS	优化后成功QPS	变化
10	99	135	+36%
50	0	109	恢复可用
100	0	0(已购)	瓶颈转移
200	0	0(已购)	瓶颈转移

限流拦截: 5,115 → 0。秒杀接口恢复可用。瓶颈从限流层转移到业务层 (uk_user_goods 防重复购买) , 这是正确的行为。

部署文件: ProductController.java , OrderController.java

部署方式: Docker Compose up -d --build backend

优化 2: /product/active 加 Redis 缓存

发现问题 (Phase 1 重测)

并发	P50(ms)	P99(ms)	QPS
50	110	168	421
100	169	315	493
200	241	462	535
500	244	1308	540

QPS 天花板停在 ~540, P99 在 c=500 时飙到 1308ms。每次请求触发 listActiveProducts() 查 DB: WHERE start_time <= now AND end_time >= now AND stock_count >= 1 ORDER BY id DESC LIMIT 5。

这是秒杀列表页——浏览商品的用户远比下单的用户多, 是真正的"高频读"场景。完全没有缓存意味着每次浏览都打 DB。

根因分析

ProductService.listActiveProducts() 直接查 DB, 无缓存层。该接口限流 1000/s (优化 1 调整后) , 在真实秒杀中, 99% 的流量是浏览行为, 不应每次穿透到 DB。

优化措施

```
// ProductService.java - listActiveProducts() 增加 Redis 缓存

private static final long ACTIVE_LIST_CACHE_TTL = 10;
private static final String ACTIVE_LIST_CACHE_KEY = "product:active:list";

public IPage<ProductVo> listActiveProducts(PageRequest pageRequest) {
    String cacheKey = ACTIVE_LIST_CACHE_KEY + ":"
        + pageRequest.getPageNum() + ":" + pageRequest.getPageSize();

    // 1. 先查 Redis 缓存
    Object cached = cacheService.get(cacheKey);
    if (cached != null) {
        return (IPage<ProductVo>) cached;
    }

    // 2. 缓存未命中--查 DB
    IPage<ProductVo> result = productMapper.selectPage(page, wrapper)
        .convert(this::convertToVo);

    // 3. 写入缓存, TTL 10s
    cacheService.set(cacheKey, result, ACTIVE_LIST_CACHE_TTL);
}
```

```

        return result;
    }

    // 4. 增删改时主动失效缓存
    private void invalidateActiveListCache() {
        cacheService.delete("product:active:list:1:5");
        cacheService.delete("product:active:list:1:10");
        cacheService.delete("product:active:list:1:20");
        cacheService.delete("product:active:list:1:50");
    }

```

缓存策略选择理由：

- TTL 10s：秒杀期间库存变化快，10s 内的数据延迟对浏览列表影响可忽略
- 主动失效 + TTL 兜底：增删改立刻失效；即使失效遗漏，10s 后自动过期
- 按 pageNum+pageSize 分 key：不同分页参数不互相污染

重测验证

首次调用 (cache miss) : 0.123s
 二次调用 (cache hit) : 0.023s ← 5.3x 加速

Redis 缓存 TTL: 10s ← 正确

部署文件: ProductService.java

部署方式: Docker Compose `up -d --build backend`

优化 3：Redis 库存自动预热

发现问题

在 Phase 1 首次压测前，发现 Redis 中**没有任何** `product:stock:*` key。库存扣减流程 `OrderService.decreaseStockWithCache()` 在 cache 为 null 时跳过 Redis Lua 直接走 DB，导致高并发下：

1. Redis 原子扣减形同虚设
2. 每次下单直接穿透到 DB 做 `UPDATE stock_count = stock_count - 1`
3. DB 行锁竞争加剧，吞吐下降

根因分析

现有代码仅在 `addProduct()`、`updateProduct()`、`getStock()` 被调用时才写入 Redis 库存。但初始 20 个商品是 SQL 脚本直接 INSERT 的——这些商品永远不会触发以上方法，导致 Redis 永远没有它们的库存。

优化措施

```

// ProductService.java - 应用启动时自动预热

@PostConstruct
public void warmupStock() {
    log.info("开始Redis库存预热...");
    List<Product> products = productMapper.selectList(null);
    int count = 0;

```

```
for (Product p : products) {
    if (p.getStatus() != null && p.getStatus() != 3) {
        cacheService.setProductStock(p.getId(), p.getStockCount());
        count++;
    }
}
log.info("Redis库存预热完成, 共预热 {} 个商品", count);
}
```

关键设计决策:

- `@PostConstruct` 确保在依赖注入完成后、首次请求到达前执行
- 过滤 `status=3` (已下架) 商品, 不预热无效库存
- 跨模块无依赖——仅使用已有的 `productMapper` 和 `cacheService`

验证

启动日志:

2026-06-21T17:40:29 INFO - 开始Redis库存预热...

2026-06-21T17:40:30 INFO - Redis库存预热完成, 共预热 20 个商品

Redis 验证:

> GET product:stock:1

"75" ← 与 DB stock_count 一致

部署文件: `ProductService.java`

部署方式: Docker Compose `up -d --build backend`

优化效果汇总

指标	优化前	优化后	改善
<code>/order</code> 限流拦截 (c=50)	100%	0%	∞
<code>/order</code> 实际可用 QPS	0 (@c≥50)	135 (@c=10)	恢复可用
<code>/product/active</code> P99 (c=500)	1308ms	预期 < 200ms (缓存命中)	~6x
<code>/product/active</code> 首次响应	123ms	123ms	—
<code>/product/active</code> 缓存命中响应	—	23ms	5.3x
Redis 库存自动预热	无 (需手动)	启动自动完成	—
Lua 原子扣减	简单 DECR (无二次校验)	checkAndDecrement (DECR+负数回滚)	防并发窗口超卖
订单初始状态	1 (已支付)	0 (待支付)	MQ 超时取消生效
MQ 超时取消	永不触发 (立即 ACK)	@Scheduled 30s 扫描	超时订单自动取消+库存恢复
超卖	0	0	不变

优化 4：热路径改用 `checkAndDecrementStock`

发现问题

`OrderService.decreaseStockwithCache()` 和 `ProductService.decreaseStock()` 都使用简单的 `decrementStock()` Lua 脚本——仅做 GET → 检查 > 0 → DECR。在极端并发下，Redis GET 和 DECR 之间仍存在极小的窗口期。

系统早已准备了更健壮的 `checkAndDecrementStock()` (对应 `check_and_decrement.lua`)：

- DECR 后再次检查结果 < 0 → 自动 INCR 回滚
- 可选传入 `expectedStock` 做乐观锁校验

但热路径从未调用过它。

根因分析

```

-- decrement_stock.lua (旧热路径)
local stock = redis.call('GET', key)
if stock <= 0 then return -2 end
redis.call('DECR', key)      -- 无法回滚
return redis.call('GET', key)

-- check_and_decrement.lua (正确方案)
local newStock = redis.call('DECR', key)
if newStock < 0 then
    redis.call('INCR', key)    -- 原子回滚!
    return 0
end
return 1

```

优化措施

```

// OrderService.java:116-139 - 用 checkAndDecrementStock 替代 decrementStock

private boolean decreaseStockWithCache(Long productId) {
    Integer cachedStock = cacheService.getProductStock(productId);

    if (cachedStock != null) {
        // 改用带双重校验的原子操作
        int luaResult = cacheService.checkAndDecrementStock(productId);
        if (luaResult != 1) {           // 1=成功, 0=库存不足, -1=key不存在, -2=冲突
            log.warn("[Lua扣减失败] productId={}, result={}", productId,
luaResult);
            return false;
        }
    }

    int dbResult = productMapper.decreaseStock(productId);
    if (dbResult <= 0) {
        if (cachedStock != null) {
            cacheService.incrementStock(productId); // DB失败回滚Redis
        }
        return false;
    }
    return true;
}

```

`ProductService.decreaseStock()` 同步修改。

验证

后台日志：
 2026-06-21T17:48:39 INFO - [Lua扣减成功] productId=10
 库存: Redis:64→63, DB:64→63 (一致)

优化 5：修复订单初始状态 (1→0 待支付)

发现问题

`OrderService.java:82` — `order.setStatus(1)` 直接将订单标记为"已支付", 跳过了"待支付(0)"状态。

导致两个连锁问题:

1. **MQ 超时取消失效**: `OrderConsumer.handleOrderTimeout()` 只在 `status==0` 时取消订单+恢复库存。 `status=1` 的订单永远不会被自动取消。
2. `OrderConsumer.handleOrderCreate()`: 检查 `status != 0` 后跳过处理, 创建订单后的异步逻辑 (如发送通知等) 被跳过。

优化措施

```
// OrderService.java:82
- order.setStatus(1);    // 直接已支付
+ order.setStatus(0);    // 待支付→等用户支付或15分钟超时自动取消
```

验证

API 响应: `"status":0, "payExpireTime":1782036219 ← createTime + 900s`

DB 查询:

`id=257, status=0, ctime=2026-06-21 17:48:39` ☒

新订单 `status=0`, 旧订单 (`status=1`) 不受影响。

优化 6: MQ 超时订单定时扫描取消

发现问题 (2026-06-22 Phase 3 长时压测)

Phase 3 持续 25 分钟压测后, MQ 架构审查发现 `order.dead.letter.queue` 全程 **0 条消息**, 但 1498/1500 笔订单的 `payExpireTime` (15 分钟) 已过期仍停留在 `status=0` (待支付)。

MQ 超时取消机制形同虚设——死信队列从未触发。

根因分析

```
order.create.queue (TTL=10min)
|
├─ OrderConsumer.handleOrderCreate() ← 立即消费并 ACK (停留 <1ms)
|
└─ (过期消息) → order.dead.letter.queue ← 永不触发
```

关键逻辑链: `OrderService.createOrder()` 发送消息到 `order.create.queue` → `OrderConsumer.handleOrderCreate()` **立即 ACK** → 消息在队列停留时间 `<1ms` → `x-message-ttl=600000` 永远不会到期 → 死信队列永远为空。

对比 `handleOrderCreate` 的设计意图: 它本应在 `payExpireTime` 之后再处理 (利用 TTL 延迟), 但实现上立即 ACK 导致 TTL 失效。

影响

影响项	严重度	说明
超时订单不取消	严重	<code>payExpireTime</code> 过期后订单仍为 <code>status=0</code> ，永久停留
库存不恢复	严重	被锁定的库存（Redis + DB）永不释放，其他用户无法购买
死信队列闲置	中等	<code>handleOrderTimeout()</code> 代码正确但从不执行

优化措施

新增 `@Scheduled` 定时扫描，每 30 秒扫描 `payExpireTime < now AND status=0` 的订单并取消，绕过不工作的 TTL 机制：

```
// OrderService.java - 新增定时扫描方法

@Scheduled(fixedRate = 30000)
public void cancelExpiredOrders() {
    long now = System.currentTimeMillis() / 1000;
    List<Order> expiredOrders = orderMapper.selectExpiredOrders(now, 100);

    if (expiredOrders.isEmpty()) {
        return;
    }

    log.info("[超时扫描] 发现 {} 个超时订单，开始取消", expiredOrders.size());
    int cancelled = 0;

    for (Order order : expiredOrders) {
        try {
            order.setStatus(2);
            orderMapper.updateById(order);

            productMapper.incrementStock(order.getGoodsId()); // 恢复 DB 库存
            cacheService.incrementStock(order.getGoodsId()); // 恢复 Redis 库存

            cacheService.deleteOrder(order.getId()); // 清理缓存
            cacheService.deleteProduct(order.getGoodsId());

            cancelled++;
        } catch (Exception e) {
            log.error("[超时取消失败] orderId={}", order.getId(), e);
        }
    }

    log.info("[超时扫描] 完成，成功取消 {}/{ 个订单", cancelled,
        expiredOrders.size());
}
```



```
// OrderMapper.java - 新增查询方法

@Select("SELECT * FROM seckill_order WHERE status = 0 AND pay_expire_time < #
{now} "
      + "ORDER BY pay_expire_time ASC LIMIT #{limit}")
List<Order> selectExpiredOrders(@Param("now") Long now, @Param("limit") int
limit);
```

设计决策：

- **fixedRate=30s**：15 分钟支付窗口 + 30 秒扫描间隔 → 最长延迟 30 秒，可接受
- **每批 limit=100**：防止一次性加载过多订单导致内存压力
- **逐条 try-catch**：单条失败不影响其他订单取消
- **DB + Redis 双恢复**：productMapper.incrementStock 恢复 DB 库存 +
cacheService.incrementStock 恢复 Redis 库存（之前 cancelOrder() 只恢复 Redis，
handleOrderTimeout() 只恢复 DB——此处统一同时恢复）
- **@EnableScheduling**：已在 MvpApplication.java:9 启用，无需额外配置

验证结果 (2026-06-22 线上验证)

1. 部署后首次扫描（启动时）

后台重启后，@Scheduled 立即扫描并取消了 Phase 3 遗留的 100 个超时订单：

```
[超时扫描] 发现 100 个超时订单，开始取消
[超时取消] orderId=1820, goodsId=23, userId=792
[库存恢复] Lua脚本执行成功 | 商品ID：23，当前库存：9524
... (x100)
[超时扫描] 完成，成功取消 100/100 个订单
```

2. 针对性验证

手动将 10 个新压测创建的订单 payExpireTime 设为过去，35 秒后验证：

检查项	优化前	优化后
过期订单状态	status=0（永不取消）	status=2（已取消）
取消率	0/1498	10/10
DB 库存恢复	✗ 不变	☑ pid=21: +2, pid=22: +4, pid=23: +4
Redis 库存恢复	✗ 不变	☑ 与 DB 同步恢复
超时→取消延迟	∞	≤ 30s

3. 压测后数据一致性

优化后轻量压测（c=50→100→200→500，1min/级）：

指标	优化前 (Phase 3)	优化后	变化
总请求数	~1,055K	193,159	—
总订单	1,500	1,068	—
超卖	0	0	不变
重复订单	0	0	不变
Redis/DB 偏差	0	0	不变
c=50 active P99	221ms	192ms	略优
c=100 active P99	453ms	425ms	略优
c=200 active P99	988ms	931ms	略优
c=500 active P99	2300ms	1619ms	~30%↓
c=500 order P99	2270ms	1639ms	~28%↓

P99 改善主要因 Phase 3 是 2min/级（累计竞争时间更长），而非本次代码优化直接导致。

4. 日志确认

```
2026-06-22T01:31:47 INFO [超时扫描] 发现 10 个超时订单，开始取消
2026-06-22T01:31:47 INFO [超时取消] orderId=3258, goodsId=21, userId=773
... (x10)
2026-06-22T01:31:47 INFO [超时扫描] 完成，成功取消 10/10 个订单
```

部署文件:

- `OrderService.java` (新增 `cancelExpiredOrders()` + `List` + `Scheduled` import)
- `OrderMapper.java` (新增 `selectExpiredOrders()` + `List` import)

部署方式: Docker Compose `up -d --build backend`

待优化项

项目	优先级	说明
<code>/product/active</code> 缓存启动预填充	P2	首次请求仍是 cache miss, 可 <code>@PostConstruct</code> 预热
限流器改为真正滑动窗口	P3	当前 INCR+固定 TTL 存在边界误杀可能
Snowflake ID 前端精度丢失	P3	已用 <code>json-bigint</code> 处理, 需确认所有接口覆盖
<code>pay_expire_time</code> 字段加索引	P2	<code>selectExpiredOrders</code> 查询 <code>WHERE status=0 AND pay_expire_time < ?</code> , 无复合索引时随表增长变慢

